

Tank Filling and Mixing System

Industrial Automation Project

1. Project Overview

Tank Filling and Mixing Control System

This project involved the design and implementation of a **simulated three-stage tank filling and mixing system** using industrial Programmable Logic Controller (PLC) programming and Human-Machine Interface (HMI) design. The system was developed to demonstrate how a basic industrial process can be automatically controlled, monitored, and organized using methods commonly found in manufacturing and process-control applications.

The automated sequence consists of three main operating stages: **Fill, Mix, and Drain**. During the filling stage, the inlet valve is activated until the tank reaches its high-level condition. The system then transitions to the mixing stage, where the mixer motor operates for a defined period using a PLC timer. After mixing is complete, the discharge valve opens and the tank drains until the low-level condition is reached. The sequence can then return to its ready state for another operating cycle. This follows the state-based sequence required by the project specification.

Objective

The objective of the project was to develop a small but complete example of an **industrial automation system**, rather than simply demonstrate individual ladder-logic instructions. The project combines sensor inputs, output devices, timer logic, operating states, start/stop control, and an operator interface into one organized control system.

The PLC program was structured so that each stage of the process has a clearly defined purpose and transition condition. Descriptive variable names and rung comments were also used to make the program easier to read, troubleshoot, and maintain. These design choices support the project's emphasis on practical implementation, structured troubleshooting, and professional documentation.

Software

The project was developed using **Rockwell Automation Connected Components Workbench (CCW)**. CCW provides the programming environment used to configure the controller, create PLC variables, develop the ladder-logic program, and configure the associated operator interface.

Ladder Logic was selected because it is widely used for industrial machine and process control and provides a visual representation of control conditions and outputs.

Controller

The control program was designed around a **Rockwell Automation Micro850 controller architecture**. The PLC program contains the logic responsible for coordinating the process sequence, including:

- Start and stop control
- Ready, Fill, Mix, and Drain operating states

- High- and low-level sensor conditions
- Inlet and discharge valve control
- Mixer motor control
- Mixing timer operation
- Process status and fault information

The program uses a state-based structure so that the equipment operates according to the current stage of the process. For example, the inlet valve is associated with the Fill state, while the mixer motor and timer are associated with the Mix state. This makes the sequence easier to follow and provides a clear troubleshooting path if the process does not advance as expected.

HMI Platform

The operator interface was designed in **CCW for a PanelView 800 (PV800) HMI architecture**. The HMI provides a visual representation of the tank process and connects operator controls and status displays to PLC variables through configured HMI tags.

The main operator screen includes a graphical tank and level indication, Start and Stop controls, Fill/Mix/Drain state indicators, inlet and discharge valve status, mixer status, timer information, system status, and an alarm banner. These elements were selected to give the operator the most important process information without unnecessary visual clutter, consistent with the project's requirement for a minimalist, readable, and operationally practical interface.

Together, the PLC program and HMI demonstrate the basic relationship found in a real automated system: **the PLC performs the control logic, while the HMI allows an operator to interact with and monitor that logic**. The completed project therefore demonstrates not only ladder-logic programming, but the integration of sequencing, field-device logic, operator controls, process visualization, and troubleshooting-oriented design into a single automation application.

2. Functional Requirements

The Tank Filling and Mixing System was designed around a set of functional requirements that define how the process should start, operate, transition between stages, and return to its initial condition. The control logic was organized so that each major function could be clearly identified within the PLC program and represented on the operator interface.

Start/Stop Control

The system includes a **Start/Stop latch** that controls whether the automatic sequence is permitted to run. Pressing the Start command establishes the system run condition and allows the process sequence to begin. The run condition remains active after the Start command is released.

The Stop command removes the run condition and prevents the automatic sequence from continuing. This provides a clear master control for starting and stopping the simulated process.

Ready State

A dedicated **Ready state** establishes the starting condition of the sequence. When the system is running and no Fill, Mix, or Drain state is active, the controller places the process in Ready.

This gives the sequence a defined starting point rather than allowing the process to enter an operating stage without an established state.

Filling Sequence

From Ready, the system transitions into the **Fill state**. During this stage, the PLC commands the inlet valve to open, allowing the simulated tank to fill.

The Fill state remains active until the required tank-level condition is detected. This demonstrates the use of a process input to determine when an operation has been completed rather than relying only on a fixed time period.

High-Level Transition

The **High Level sensor** provides the transition condition between filling and mixing. When the tank reaches its high-level condition, the PLC ends the Fill state and transitions the sequence into the Mix state.

At this point, the inlet valve is no longer required for filling, and the next stage of the process can begin. This creates a clear relationship between the simulated field sensor and the PLC sequence logic.

30-Second Mixing Timer

During the **Mix state**, the mixer motor is commanded to operate and a PLC timer controls the required mixing period. The project specification defines this period as **30 seconds**.

The timer allows the mixer to operate for a controlled duration before the sequence can advance. Once the timer reaches its preset value, its completion condition is used to transition the process from Mix to Drain.

Drain Sequence

After mixing is complete, the system enters the **Drain state**. During this stage, the discharge valve is commanded open so that the contents of the simulated tank can be discharged.

The drain operation continues until the **Low Level sensor** indicates that the tank has reached its required low-level condition. The Low Level input therefore acts as the completion condition for the discharge stage.

Automatic Reset

Once the Low Level condition is reached, the Drain state is cleared and the sequence returns to its initial operating condition. This allows the process to prepare for another cycle without requiring the individual sequence states to be manually reset.

The resulting automatic sequence follows a clear operating path:

Ready → Fill → Mix → Drain → Ready

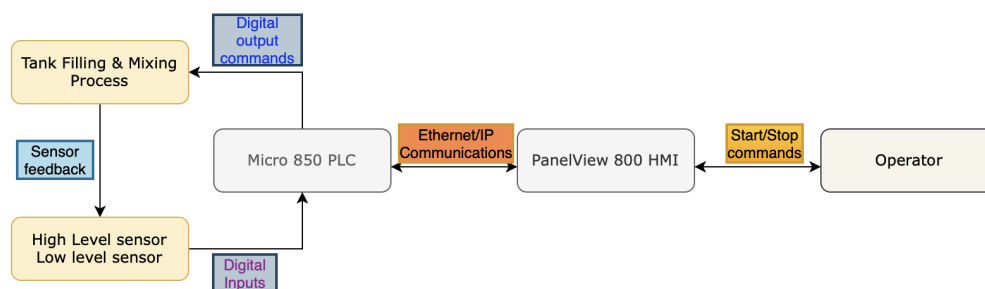
This state-based approach makes the process easier to understand and provides a logical troubleshooting path because each stage has a defined operating condition and a defined transition condition.

HMI Operator Interface

An **HMI operator interface** was developed to provide control and visual monitoring of the automated process. The interface includes Start and Stop controls, process-state indication, tank-level visualization, valve and mixer status, timer information, system status, and alarm indication.

HMI tags were mapped to the corresponding PLC variables so that the interface reflects the structure of the control program. The HMI was designed to keep important operating information visible on a single main screen while maintaining a simple and organized layout. This supports the requirement for an interface that is minimalist, professional, easy to interpret, and operationally practical.

3. Control System Architecture



The control system architecture is organized around the relationship between the operator, the HMI, the PLC, and the simulated tank process. The operator enters Start and Stop commands through the PanelView 800 HMI, which is designed to exchange control and status information with the Micro850 PLC over an EtherNet/IP communication network. The PLC acts as the main controller by processing operator commands, level-sensor conditions, timer values, and the current operating state. Based on this information, it controls the inlet valve, mixer motor, and discharge valve according to the programmed Fill, Mix, and Drain sequence. Process conditions are then returned to the PLC through the input signals and displayed on the HMI, creating a continuous feedback loop that allows the operator to monitor the system as it operates.

This arrangement reflects a typical industrial control architecture in which the HMI provides operator interaction, the PLC performs the control logic, and the field inputs and outputs connect the controller to the physical process.

4. Tag Philosophy

The PLC program uses a **consistent tag-naming structure** so that each variable can be understood quickly without needing to trace the entire program. Tag names describe both the function of the variable and its role within the control system.

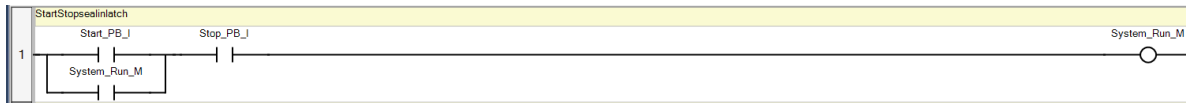
The naming convention uses suffixes such as `_I` for input signals, `_O` for controlled outputs, and `_M` for internal memory or status bits. State variables also use descriptive names such as `State_Fill_M` and `State_Mix_M`, making the current operating stage easy to identify during troubleshooting.

Tag	Purpose
Start_PB_I	Operator Start push-button input
Stop_PB_I	Operator Stop push-button input
High_Level_I	High-level tank sensor used to complete filling
Low_Level_I	Low-level tank sensor used to complete draining
High_Pressure_I	High-pressure fault input
Inlet_Valve_O	Output controlling the inlet valve
Mixer_Motor_O	Output controlling the mixer motor
Discharge_Valve_O	Output controlling the discharge valve
System_Run_M	Internal system run status
State_Ready_M	Identifies the Ready state
State_Fill_M	Identifies the Fill state
State_Mix_M	Identifies the Mix state
State_Drain_M	Identifies the Drain state
Mix_Timer	TON timer used to control the mixing period

This naming approach improves **readability, maintenance, and troubleshooting** because the purpose of each variable is visible directly from its name and description. It also supports the project requirement that PLC tags and variables use clear, professional descriptions rather than generic or unclear identifiers.

5. PLC Design and Implementation

Operator Controls



The operator interface uses a traditional **Start/Stop seal-in circuit** to control the main system run command.

When the operator presses the **Start** button, the `Start_PB_I` input energizes the `System_Run_M` memory bit. A parallel `System_Run_M` contact then maintains the run command after the Start button is released. This allows the system to remain active without requiring the operator to continuously hold the Start button.

The Stop command is placed in series with the run logic. When the Stop condition is activated, the seal-in circuit is broken and `System_Run_M` is de-energized. This removes the main run permission used by the remaining sequence logic.

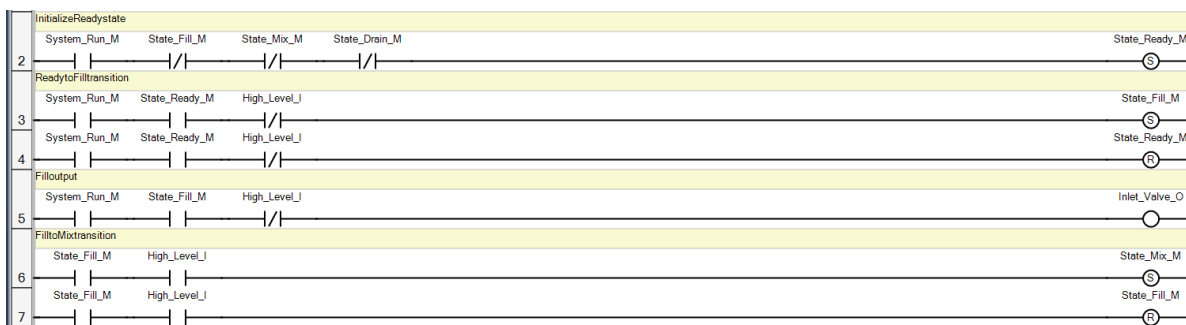
Using a dedicated system run bit provides a clear separation between **operator commands** and the **automatic process sequence**. The Fill, Mix, and Drain stages can therefore use `System_Run_M` as a common operating condition, making the ladder logic easier to understand and troubleshoot.

Control sequence:

Start Command → `System_Run_M` Latched → Automatic Sequence Enabled → Stop Command → `System_Run_M` Cleared

This structure provides a simple and recognizable method of machine control while supporting the project's goal of readable, maintainable ladder logic and clear troubleshooting flow.

State Machine



The process sequence is controlled using dedicated **state memory bits** representing the Ready, Fill, Mix, and Drain stages. These states determine which part of the process is permitted to operate at each point in the sequence.

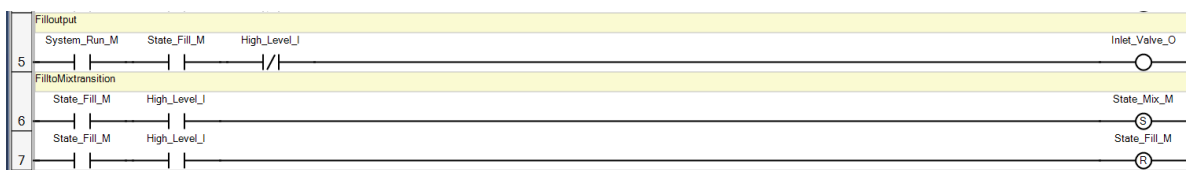
The logic is structured so that the process progresses in a defined order:

Ready → Fill → Mix → Drain → Ready

Each state remains active until its required transition condition is satisfied. For example, the Fill state transitions to Mix when the High Level sensor is activated, while the Mix state transitions to Drain when the mixing timer is complete.

The state logic is designed so that **only one operating state is active at a time**. This prevents conflicting stages from operating simultaneously and creates a clear sequence that is easier to follow, maintain, and troubleshoot. This state-machine structure directly supports the sequencing requirement defined for the project.

Fill Routine

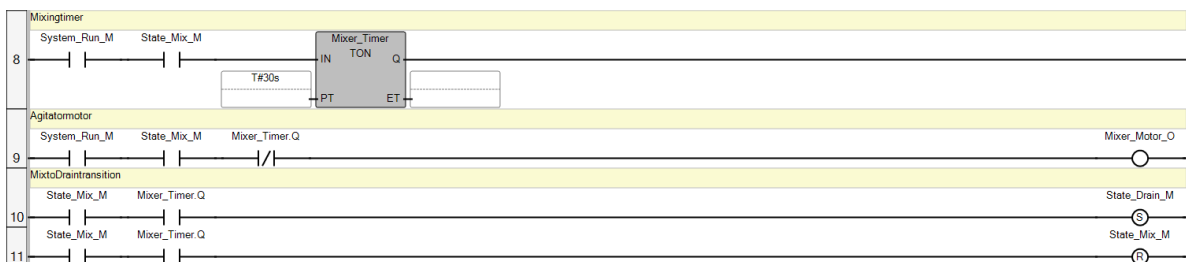


The **Fill routine** controls the first active stage of the process. While the State_Fill_M state is active and the system is running, the PLC energizes the Inlet_Valve_O output to begin filling the tank.

The inlet valve remains energized until the High_Level_I sensor becomes TRUE, indicating that the required tank level has been reached. The inlet valve is then de-energized, the Fill state is cleared, and control transfers to the **Mix state**.

This allows the filling operation to be controlled directly by tank level rather than by a fixed time period.

Mixing Routine

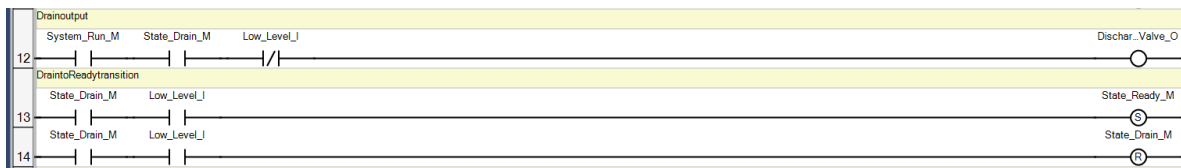


The **Mixing routine** begins after the Fill stage has been completed. While State_Mix_M is active, the PLC enables the mixer sequence and starts the Mix_Timer **TON (Timer On Delay)** instruction.

The timer is configured for the required **30-second mixing interval**. During this period, the Mixer_Motor_O output remains energized, allowing the agitator motor to operate for the specified amount of time.

When the timer reaches its preset value, the timer completion bit becomes TRUE. The mixer motor is stopped, the Mix state is cleared, and control transfers to the **Drain state**. This provides a repeatable, time-controlled mixing operation as required by the project specification.

Drain Routine



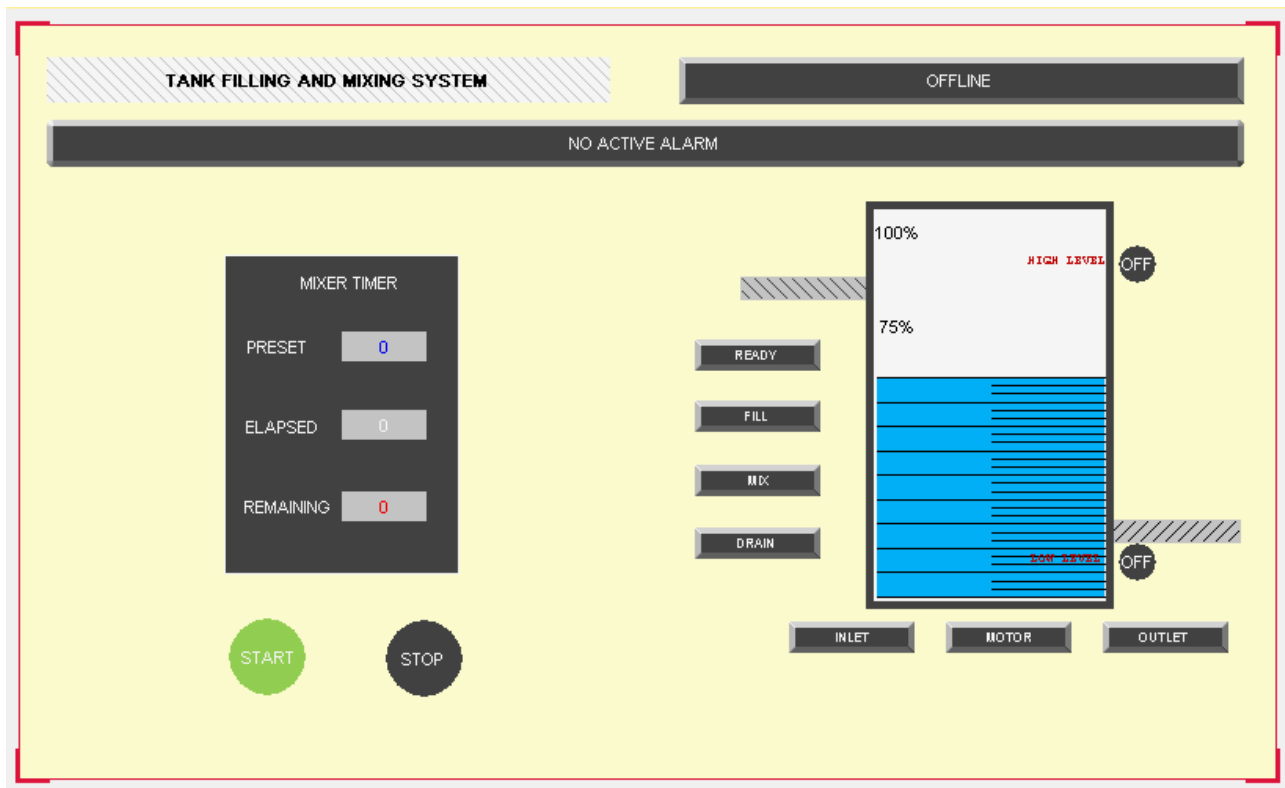
The **Drain routine** controls the final active stage of the process. While State_Drain_M is active and the system is running, the PLC energizes the Discharge_Valve_O output to begin emptying the tank.

The discharge valve remains energized until the Low_Level_I sensor indicates that the tank has reached its required low-level condition. Once this condition is detected, the discharge valve is de-energized and the Drain state is cleared.

The sequence then returns to the **Ready state**, placing the system in its initial operating condition and preparing it for the next cycle.

Fill → High Level → Mix → Timer Complete → Drain → Low Level → Ready

6. HMI Design



The HMI was designed to give the operator a **clear view of the process at a glance**. Rather than filling the screen with unnecessary graphics, the layout groups controls, process information, equipment status, and alarms into separate visual areas. This supports the project's goal of creating a minimalist and operationally practical interface that prioritizes operator readability and situational awareness.

Operator Controls — Start / Stop

The **Start and Stop controls** are grouped together in the lower-left portion of the screen. They are separated from the process equipment so that operator commands can be quickly distinguished from status information.

Keeping these two controls together also creates a predictable control area: the operator knows where to look when direct interaction with the process is required.

Tank and Process Visualization

The **tank is the largest process element** and is positioned prominently on the right side of the interface. This was intentional because the tank is the central piece of equipment in the Fill–Mix–Drain process.

The **tank-level visualization** is incorporated directly into the tank graphic instead of being displayed elsewhere on the screen. This allows the operator to associate level information immediately with the

equipment it represents. High- and low-level indications are positioned beside the tank for the same reason.

Valve and Motor Indicators

The **inlet valve, mixer motor, and outlet valve** indicators are arranged directly beneath the tank. Their placement reflects their relationship to the process equipment rather than presenting them as unrelated status values elsewhere on the screen.

This creates a simple visual relationship between the tank and the equipment responsible for filling, mixing, and draining it. The Ready, Fill, Mix, and Drain indicators are also grouped vertically beside the tank so that the current sequence stage can be identified without searching across the display.

Alarm Banner

The **alarm banner is positioned across the top of the HMI**, where it remains highly visible regardless of which process stage is active. Alarm information is intentionally separated from routine process information so that abnormal conditions can attract the operator's attention quickly.

Placing the alarm area near the top also establishes a clear information hierarchy: abnormal system conditions receive priority, while normal process details remain available below. This follows the requirement to emphasize alarm awareness without introducing unnecessary visual clutter.

Overall Design Approach

The interface follows a simple visual hierarchy:

System and Alarm Status → Process Visualization → Equipment Status → Operator Controls

The layout was intentionally kept restrained, with large process elements, clearly separated information areas, and limited decorative graphics. The goal was not to make the HMI visually elaborate, but to make important information **easy to locate and interpret quickly**.

By grouping related information and keeping the tank as the main visual focus, the interface supports operator situational awareness while maintaining a clean and professional industrial appearance.

7. Validation

PLC Online — Fill State Active



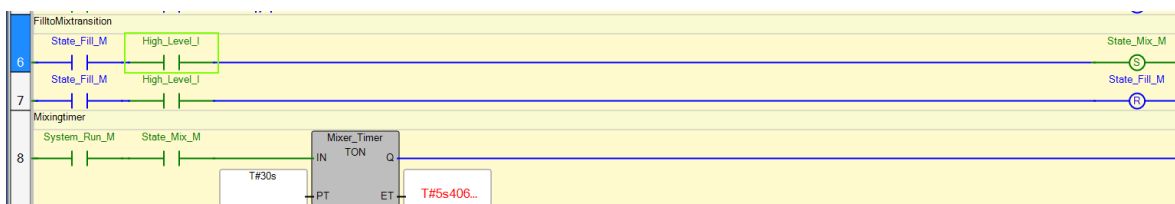
The first validation screenshot demonstrates the PLC operating in the **Fill state** while connected to the simulator. The ladder logic shows active power flow through the Fill routine, with `State_Fill_M` energized and the `Inlet_Valve_O` output commanded ON. At this stage, the `High_Level_I` sensor remains FALSE, allowing the filling operation to continue.

This screenshot confirms that the PLC is operating online, the Fill state has been successfully entered, and the inlet valve output is energized under the correct operating conditions.

Validation demonstrates:

- PLC online communication
- Fill state active
- Inlet valve output energized

State Transition — Fill → Mix



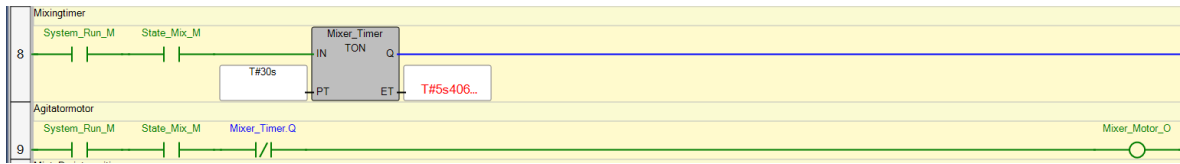
The second validation screenshot captures the transition between the **Fill** and **Mix** states. Once the `High_Level_I` sensor becomes TRUE, the Fill state is cleared and the Mix state is activated. At the same time, the mixing timer begins its timing sequence.

This demonstrates that the state-machine logic correctly advances from one operating stage to the next based on the required process condition rather than operator intervention.

Validation demonstrates:

- High Level transition detected
- Fill state deactivated
- Mix state activated
- Automatic sequence progression

Mixing Timer



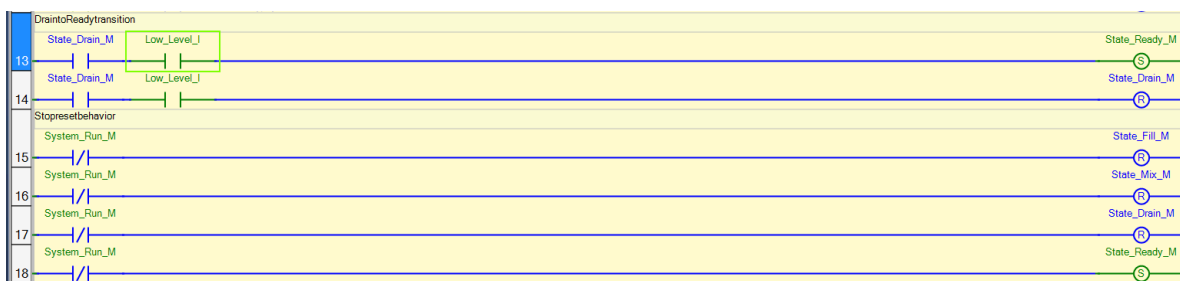
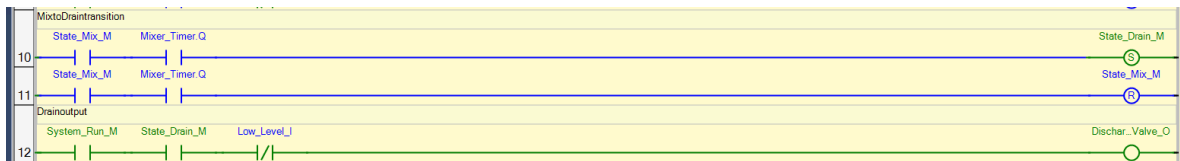
The third validation screenshot shows the system operating in the **Mix state** while the TON instruction controls the required mixing interval. The elapsed time (ET) increases toward the preset value while the Mixer_Motor_O output remains energized.

This confirms that the timer and output operate together throughout the mixing cycle and that the process remains in the Mix state until the timer has completed.

Validation demonstrates:

- TON timer operating correctly
- Elapsed time counting upward
- Mix state active
- Mixer motor energized

Drain State and Sequence Reset



The final validation screenshots show the process operating in the **Drain state**. During this stage, the Discharge_Valve_O output remains energized while the tank is emptied. When the Low_Level_I

sensor becomes TRUE, the Drain state is cleared and the controller returns the sequence to the Ready state.

This final transition verifies that the complete operating sequence executes as intended and automatically prepares the system for the next operating cycle.

Validation demonstrates:

- Drain state active
- Discharge valve energized
- Low Level transition detected
- Automatic return to Ready state
- Successful sequence reset

The PLC control sequence was successfully validated using the **Micro800 Simulator**, confirming correct operation of the state machine, timer logic, and process outputs throughout the Fill, Mix, and Drain sequence. The HMI application was also successfully configured and validated within the Connected Components Workbench (CCW) development environment and prepared for deployment to a **PanelView 800** terminal. A live HMI runtime demonstration was not included in this project because CCW does not provide a desktop runtime environment for PanelView 800 applications. As a result, HMI validation was performed through interface design, *tag mapping*, and project configuration rather than live runtime execution.

8. Lessons Learned

One of the most valuable lessons from this project came from working through **software, simulation, and communication limitations**, rather than from the ladder logic itself.

During development, I initially encountered communication and simulation issues while running **Connected Components Workbench (CCW) in a virtualized environment on macOS**. Although the PLC logic and HMI could be developed, the virtualization environment introduced limitations when attempting to establish the communication required for full simulation and runtime testing.

To address this, I migrated the project to a **native Windows 11 environment**. This provided a more suitable environment for CCW and allowed the PLC simulation and HMI configuration to be validated more reliably.

This experience reinforced an important part of industrial automation work: **successful controls development depends on more than writing correct PLC logic**. Software compatibility,

communication paths, controller configuration, simulation tools, and the engineering environment itself can all affect commissioning and troubleshooting.

The project also strengthened my approach to troubleshooting. Instead of repeatedly changing working control logic to solve an external communication problem, I learned to separate **programming issues from platform and communication issues**, identify the actual source of the limitation, and adjust the development environment accordingly.

Overall, the project provided practical experience not only in PLC sequencing and HMI design, but also in the troubleshooting and problem-solving process required to move an automation project from initial development toward validation.